

BuildOrbit Enterprise Deployment & Operations Guide

This document serves as the authoritative, step-by-step guide for DevOps engineers and system administrators tasked with deploying the BuildOrbit enterprise platform into a production environment.

BuildOrbit is architected as a modern Next.js 16 (App Router) monolithic application, utilizing Prisma for robust ORM capabilities against a PostgreSQL database.

1. System Architecture & Prerequisites

Before initiating the deployment process, ensure your infrastructure meets the following minimum specifications:

- **Runtime:** Node.js v20.x (LTS) or higher.
 - **Database:** PostgreSQL v14 or higher. We highly recommend a serverless database solution such as NeonDB or Supabase for automatic connection pooling and horizontal scaling.
 - **Package Manager:** npm v10.x.
 - **External Services:**
 - **Cloudinary:** Required for managing employee avatars, medical certificates, and document uploads.
 - **Resend:** Required for transactional email delivery (e.g., Leave Approval notifications, Payroll alerts).
 - **Redis (Optional but Recommended):** For rate limiting and session caching (Upstash recommended).
-

2. Environment Configuration

BuildOrbit relies on strict environment variable configuration. You must securely inject the following variables into your production environment (e.g., via Vercel Environment Settings, AWS Secrets Manager, or a `.env.production` file).

```
# -----
# DATABASE CONFIGURATION
# -----
# The primary connection string for Prisma. Must include sslmode=require for prod.
DATABASE_URL="postgresql://username:password@your-database-host:5432/buildorbit?
sslmode=require"

# -----
# AUTHENTICATION & SECURITY
# -----
# The canonical URL of your production deployment (e.g., https://app.buildorbit.com)
NEXTAUTH_URL="https://app.buildorbit.com"
# A cryptographically secure random string (minimum 32 characters) used to sign
JWTs.
NEXTAUTH_SECRET="generate-a-secure-random-string-here"
# The secret used to encrypt generic session data.
SESSION_SECRET="generate-another-secure-string-here"
```

```
# -----  
# EXTERNAL INTEGRATIONS  
# -----  
# Cloudinary (Media Storage)  
NEXT_PUBLIC_CLOUDINARY_CLOUD_NAME="your-cloud-name"  
CLOUDINARY_API_KEY="your-api-key"  
CLOUDINARY_API_SECRET="your-api-secret"  
  
# Resend (Email Delivery)  
RESEND_API_KEY="re_your_resend_api_key_here"
```

3. Database Initialization & Migration

Do not run raw SQL scripts. BuildOrbit utilizes Prisma to strictly enforce schema integrity.

1. **Generate the Prisma Client:** This step builds the highly optimized TypeScript client tailored to your operating system.

```
npx prisma generate
```

2. **Deploy the Database Schema:** For production environments, always use `migrate deploy` to apply version-controlled migrations safely. *(Note: For initial QA/Testing environments without prior data, `npx prisma db push --accept-data-loss` may be used).*

```
npx prisma migrate deploy
```

3. **Seed Initial Configurations:** The seed script sets up fundamental roles, departments, leave types, and a Super Admin account essential for first-time login.

```
# Ensure SEED_ADMIN_EMAIL and SEED_ADMIN_PASSWORD are in your environment  
npx prisma db seed
```

4. Building the Production Application

Next.js aggressively optimizes the application during the build phase, generating static assets and pre-rendering server components.

```
# Compile the application (Turbopack is utilized under the hood)  
npm run build  
  
# Start the Node.js production server  
npm start
```

Note: If deploying to Vercel, the build and start processes are handled automatically based on the `package.json` scripts.

5. Enterprise Cron Job Automation

BuildOrbit relies on several automated background tasks to maintain accurate HR and Payroll records. You must configure an external cron trigger (such as Vercel Cron, AWS EventBridge, or a standard Linux `crontab`) to execute HTTP GET requests against the following secured API endpoints:

1. Auto-Checkout Enforcement

- **Endpoint:** GET `https://your-domain.com/api/cron/auto-checkout`
- **Frequency:** Daily at 23:59 (11:59 PM)
- **Purpose:** Automatically closes active attendance sessions for employees who forgot to check out, preventing runaway time logging.

2. Payroll Draft Generation

- **Endpoint:** GET `https://your-domain.com/api/cron/payroll-draft`
- **Frequency:** Monthly on the 25th day of the month at 02:00 AM.
- **Purpose:** Aggregates attendance data, computes deductions/allowances, and generates draft payroll records for HR review.

3. Annual Leave Carryforward

- **Endpoint:** GET `https://your-domain.com/api/cron/leave-carryforward`
- **Frequency:** Yearly on January 1st at 00:01 AM.
- **Purpose:** Calculates unused leave balances and rolls them over into the new calendar year based on the organization's policy.

6. Monitoring & Maintenance

- **Logging:** All Next.js errors are streamed to `stdout / stderr` . Ensure your hosting provider captures these logs (e.g., Datadog, AWS CloudWatch).
- **Database Backups:** Configure your PostgreSQL provider to perform automated daily point-in-time recovery (PITR) backups. BuildOrbit does not manage backups internally.